So Long, and Thanks for All the Seeds: Attacking GGM-trees in Post-quantum signatures

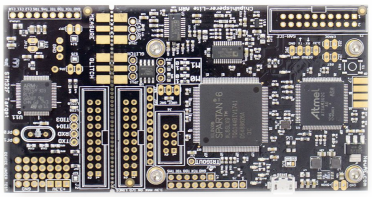
Matrix Equivalence Digital Signature

To replace classical public-key primitives (such as RSA and ECDSA) that are vulnerable to Shor's algorithm on quantum architectures, the National Institute of Standards and Technology (NIST) initiated a standardization competition to evaluate and standardize secure post-quantum digital signature schemes.
To guarantee authenticity, integrity, and non-repudiation in a post-quantum setting, several leading signature candidates rely on zero-knowledge (ZK) proof systems.
Prominent ZK-based constructions derive non-interactive digital signatures by applying the Fiat-Shamir transformation to underlying interactive identification protocols.Specifically, MEDS is a code-based post-quantum primitive whose underlying security rests on the computational hardness of the Matrix Code Equivalence (MCE) problem in the rank metric.

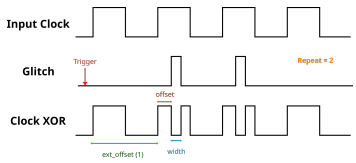
Fault Injection Attacks

Unlike passive side-channel analysis that measures physical emissions like timing or power consumption, fault-injection attacks actively disrupt physical hardware during execution.
Adversaries apply physical stressors, such as clock glitches, voltage spikes, laser pulses, or electromagnetic fields, to alter control flow, skip assembly instructions, corrupt register values, or force early loop terminations.
In seed-tree-based ZK signatures, injecting physical perturbations during seed publication can force the exposure of hidden ephemeral seeds alongside their corresponding ZK response pairs.

Our physical experimentation utilizes a CW303-STM32F3 target board featuring an ARM Cortex-M4 microcontroller (STM32F303RCT7) coupled with a ChipWhisperer-Lite capture board.

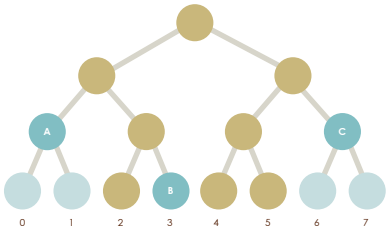


TWe choose to focus on clock glitch injections. The injection is initiated by a trigger signal preceding a defined target execution window in the code.
Glitch pulses are combined with the nominal clock via an XOR operation.
This clock perturbation disrupts processor timing, causing critical assembly instructions to be skipped or miscalculated.



GGM trees

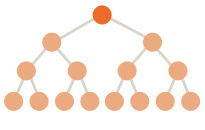
Zero-knowledge signature schemes require generating independent ephemeral commitments across multiple parallel execution rounds, each of which must be driven by an isolated random seed so that disclosing the randomness of public round does not leak information about hidden, secret-dependent rounds.
Transmitting large numbers of independent random seeds individually within every signature produces large signature payload sizes.
To minimize payload size, schemes deploy Goldreich-Goldwasser-Micali (GGM) binary trees to expand a single master root seed into all required leaf seeds via top-down pseudorandom generator evaluations.
The signer transmits the minimal set of intermediate co-path nodes (seed path) required for the verifier to recompute public leaf seeds.



To reveal the leaves {0,1,3,6,7} of this seed tree, we would only need to publish the node seeds A,B, and C to the signature.

Target faulted GGM trees

A fault model defines the targetted position in the code, the precise timing, and architectural effect of a physical disturbance on cryptographic software logic.
Our analysis targets the unified tree-traversal and path-building routine in the official C reference implementation of MEDS.
We experimented with three different models, model 1 being the most efficient as it reveals all seeds.



Model 1

By targetting the evaluation of the main loop conditional, we reveal the root nodes and all the leaf seeds



Model 2

By skipping a loop incrementation when reaching a chosen leaf, we reveal all subsequent subtrees

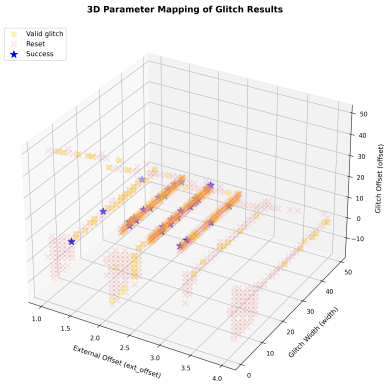


Model 3

By skipping the update of the "hidden" flag, we reveal a specific leaf.

Attack results

Model 1 demonstrated the highest success rate and displayed a broad, stable parameter region under standard compiler size optimizations.
Under dual revelation, where a faulted signature leaks a hidden leaf seed alongside its corresponding zero-knowledge response, the secret are extracted via direct matrix inversions and multiplications over $\mathbf{F}_{2}$.
Mathematical analysis proves that complete secret key recovery requires very few effective faulted signatures, with an expected query complexity of 5 across all standardized MEDS parameter sets.



Countermeasures

To mitigate seed-tree fault attacks while preserving the original signature payload size, post-publication code-path checks validate the generated path prior to transmission
Post-Publication Path Size Check C_{size}: Rejects signatures whose path is different than expected, effectively mitigating Models 1 and 3
Full Path Reconstruction Check C_{tree}: Re-expands the tree from the signature path and asserts that no hidden leaf seed is derived
Flag Tree Validation Check $Cf_{lagtree}$: Decouples flag-tree generation from path extraction, explicitly constructing and validating a binary flag tree to enforce that no ancestral node of a hidden leaf is marked as public

As shown below these solution generate very little overhead in execution time.

Parameter Set	Base Signature Time	C_{size}	C_{tree}	$C_{flagtree}$
Toy	6,845 µs	-0.77%	-0.75%	-0.15%
MEDS-9923	1,609,626 µs	-1.67%	-1.22%	-1.48%
MEDS-13220	299,950 µs	-8.02%	-5.49%	-5.49%
MEDS-41711	4,847,694 µs	+7.27%	+2.37%	+10.58%
MEDS-55604	1,280,193 µs	+7.88%	+3.22%	+2.16%
MEDS-134180	5,009,655 µs	+1.29%	+2.62%	+14.48%
MEDS-167717	2,950,702 µs	+0.04%	+4.28%	+3.42%
Toy (Cortex-M4)	753,566 µs	-2.18%	-1.95%	-2.10%